Database Migrations

FastAPI Masterclass

The Story So Far

- FastAPI created all database tables at app launch with **SQLModel.metadata.create_all(engine**).
- The database **schema** describes the database *shape*: tables, columns, types. If we update the schema (add a column/field, add a new table, etc), we delete **database.db** and restart the application.
- This design is fine for local development/experimentation but not suitable for production.

Database Migrations I

- We need to evolve the database schema over time without deleting existing data.
- A **database migration** is a Python script that describes a change to the database schema.
- A migration includes instructions on how to apply the change *and* reverse the change.

```python
def upgrade():
    with op.batch_alter_table("movies") as batch_op:
        batch_op.add_column(sa.Column("release_year", sa.Integer(),
nullable=True))

def downgrade():
    with op.batch_alter_table("movies") as batch_op:
        batch_op.drop_column("release_year")
```

Database Migrations II

- Think of a migration as instructions on how to move forwards or backwards to a previous *version*.
- Migrations are applied in order. The database keeps track of which migrations have been applied.
- Migrations provide a sequential history of how the schema evolved.

Alembic

- **Alembic** is the migration tool for SQLAlchemy (and, by extension, SQLModel).
- Alembic detects the differences between models and the current database schema, then *generates* the migrations to sync them.
- To practice, we'll build a standalone project (no FastAPI) with a few models (**Movie**, **Genre**) that will change over time.